

Figure 1.19 The same 25 x 25 `<rect>` with no, 1-px, 2-px, 3-px, 4-px, and 5-px strokes. Though these are drawn on a retina screen using half-pixels, the second and third report the same width and height (27 px x 27 px) as the fourth and fifth (29 px x 29 px).

Now you've learned why there's no SVG `<square>`. The color, stroke, and transparency of any shape can be changed by adjusting the style of the shape, with `"fill"` determining the color of the area of the shape and `"stroke"`, `"stroke-width"`, `"stroke-dasharray"` determining its outline.

Notice, though, that the inspected element has a measurement of 27 px x 27 px. That's because the 1-px stroke is drawn on the outside of the shape. That makes sense, once you know the rule, but if you change the `"stroke-width"` to `"2px"` it will still be 27 px x 27 px. That's because the stroke is drawn evenly over the inside and outside borders as seen in figure 1.19. This may not seem too big a deal, but it's something to remember when you're trying to line up your shapes later on.

Change the style parameters of the rectangle to the following:

```
"fill:purple;stroke-width:5px;stroke:cornflowerblue;"
```

Congratulations! You've now successfully visualized the complex and ambiguous phenomenon known as "ugly."

<TEXT>

SVG provides the capacity to write text as well as shapes. SVG text, though, doesn't have the formatting support found in HTML elements, and so it's primarily used for labels. If you do want to do basic formatting, you can nest `<tspan>` elements in `<text>` elements.

<G>

The `<g>` or group element is distinct from the SVG elements we've discussed in that it has no graphical representation and doesn't exist as a bounded space. Instead, it's a logical grouping of elements. You'll want to use `<g>` elements extensively when creating graphical objects that are made up of several shapes and text. For instance, if you wanted to have a circle with a label above it and move the label and the circle at the same time, then you'd place them inside a `<g>` element:

```
<g>
<circle r="2"/>
<text>This circle's Label</text>
</g>
```

Moving a `<g>` around your canvas requires you to adjust the `transform` attribute of the `<g>` element. The `transform` attribute is more intimidating than the various xy attributes of shapes, because it accepts a structured description in text of how you want to transform a shape. One of those structures is `translate()`, which accepts a pair of coordinates that